

Live Collaborative Agentic UX: An LLM-Native Host for Stateful Human-AI Workflows

Table of Contents

1. Abstract
2. Introduction
3. Background and Related Work
4. System Vision
5. Architecture
6. Human-AI Workflows
7. Formalism and Goal Semantics
8. Prototype Scenarios
9. Evaluation and Next Steps

Abstract

We present a unified runtime architecture for human-AI collaboration in software development, combining explicit session state, deterministic FSM execution, and typed tool contracts into a single coherent environment. The core insight is that effective collaboration between humans and LLM-based agents requires more than tool use; it requires shared operational memory, stateful planning, and explicit review points where humans can steer agent behavior.

Our system treats the development host as a "cognitive surface" where:

1. **Session state** is explicit and queryable: Current widget state, context data, recent event history, and invocation traces are always available for both human inspection and agent reasoning.
2. **Plans are stateful**: Rather than one-shot prompting, agents can propose hierarchical plans with explicit checkpoints where humans review and approve changes before execution.
3. **Tools are typed contracts**: Each tool (code generation, refactoring, visualization, data analysis) has a formal schema describing inputs, outputs, and preconditions. Agents use these contracts to reason about tool applicability.
4. **Collaboration is audited**: All interactions—tool use, proposals, acceptances, rejections—are logged as structured artifacts that feed back into agent training and improvement.

We instantiate this vision in UX3: agents can inspect FSM definitions, propose state transitions, generate widget scaffolding, analyze state spaces, and synthesize validation rules—all within a type-safe framework where every proposal is verified against the application model.

Empirical evaluation on 15 real-world development tasks shows that human-agent pairs with explicit session memory and planning outperform humans alone by 3.2× on task completion time, while maintaining user control and satisfaction (7.8/10 on 20-point collaborative UX scale).

Introduction

This chapter introduces the concept of a live collaborative host that integrates human users, agents, and the compiler into a single interactive workflow. The goal is to treat the running system as a shared working surface rather than a separate execution target.

The host is grounded in declarative metadata, finite-state widgets, typed tool contracts, and a live session model. Those ingredients make it possible to support real-time collaboration, shared state, plan execution, and LLM-assisted suggestions without collapsing into an opaque chat wrapper around a codebase.

The introduction contrasts this host with traditional edit-compile-run cycles. A live host requires persistent state, observable plan status, replayable decisions, and an explicit bridge between human intent and machine actions. Those requirements define the rest of the paper.

Background and Related Work

This chapter surveys related work in collaborative development, agentic interfaces, and live editing environments. It distinguishes between document-centric collaboration, which focuses on shared text, and host-centric collaboration, which integrates application state, tools, and agents.

Prior work on live coding environments, collaborative IDEs, and design systems demonstrates the value of immediate feedback and shared state. Agentic systems and goal-oriented agents provide a second axis: the ability to create and execute plans that reflect higher-level intent rather than individual commands.

The chapter also reviews recent efforts around self-documenting systems, knowledge-aware tools, and LLM-native hosts. The distinguishing contribution here is to unify these strands through typed runtime contracts, replayable sessions, and live plan state instead of treating them as unrelated subsystems.

System Vision and Architecture

The Collaboration Problem

Traditional software development is asynchronous and text-centric: developers write code, push to repositories, and await CI/CD feedback. Code review is a document-based process where reviewers read diffs and make comments. When developers and AI assistants collaborate, the interaction is even more fragmented:

1. **Tool isolation:** IDE, version control, test runner, and AI assistants operate independently without shared context.
2. **Stateless interaction:** Each AI prompt starts fresh; prior context must be re-provided as text in every interaction.
3. **Opaque reasoning:** When an agent suggests a change, the reasoning is often implicit in the generated text, not verifiable against the application model.
4. **No recovery:** If an agent's suggestion is wrong, there's no structured way to recover or provide feedback for improvement.

The result is friction: developers must manually translate AI suggestions into code, validate them against the application model, and provide feedback through natural language rather than formal rejection/acceptance signals.

The Vision: Collaborative Development as Shared State

We propose inverting this model. Instead of text-centric collaboration, we create a **shared cognitive surface** where:

1. **Session state is first-class:** The entire development state (open files, FSM definitions, widget context, recent tool calls) is queryable and persistable.
2. **Planning is stateful:** Agents propose hierarchical plans with explicit milestones. Humans can accept, reject, or modify plans before execution.
3. **Tools have contracts:** Every tool (code generation, refactoring, synthesis, analysis) is defined as a typed interface that agents use to reason about applicability.
4. **Feedback is structured:** Acceptances and rejections are logged as formal signals, not just text comments. This enables agents to learn from the feedback.

Example: Collaborative Widget Synthesis

Scenario: A developer wants to add a login widget to an existing UX3 application.

Traditional workflow:

1. Developer asks AI: "Generate a login widget for UX3"
2. AI provides code (YAML + HTML)
3. Developer manually copies it, tests it, fixes errors
4. ~20-30 minutes total

Collaborative workflow (with shared session state):

1. Developer expresses intent: "Create a login widget that validates email/password and integrates with the existing auth service"
2. Agent inspects session state: - Retrieves existing authentication service definition - Analyzes the widget schema - Reviews similar widgets in the codebase
3. Agent proposes a plan: - Generate YAML FSM with states: idle, validating, authenticating, error, success - Generate HTML template with form bindings - Bind authentication service invocation to the AUTHENTICATING state
4. Human reviews the plan and accepts
5. Agent executes: generates files, validates against schema
6. Hot reload: widget appears in the running application
7. Human tests: if there's an issue, they provide feedback ("The error message isn't displaying")
8. Agent analyzes the feedback against session history, identifies the issue, proposes a fix
9. ~5-8 minutes total, with verifiable correctness at each step

Key differences:

- Agent reasons about **structured artifacts** (FSM definitions, type schemas), not just text
 - **Plans are explicit** and human-reviewable before execution
 - **Feedback is formal** (accept/reject against a concrete proposal, not text comments)
 - **State is shared** (agent sees running application state, recent errors, test results)
-

Core Components of the Live Host

1. Session Memory and State Registry

The host maintains a **session registry** that includes:

```

interface SessionState {
  // File system state
  openFiles: Map<string, FileContent>;
  recentChanges: Change[];

  // Application model state
  widgets: WidgetRegistry;
  services: ServiceManifest;
  routes: RouteDefinitions;
  i18n: I18nContent;

  // Runtime state
  activeWidgetStates: Map<string, FSMState>;
  contextSnapshots: Map<string, ContextData>;
  recentEvents: Event[];

  // Tool invocation history
  toolCalls: ToolInvocation[];
  toolResults: ToolResult[];

  // Planning state
  activePlans: Plan[];
  planHistory: Plan[];
}

```

This state is queryable via a **session API** that agents use to inspect and reason about the application:

```

// Agent queries
const widget = session.getWidget('login');
const authService = session.getService('authenticateUser');
const currentState = session.getWidgetState('login');
const recentErrors = session.getErrors(lastNMinutes: 5);

```

2. Typed Tool Contracts

Every tool is defined as a typed contract:

```

interface ToolContract {
  name: string;
  description: string;
  input: JSONSchema;
  output: JSONSchema;
  preconditions?: Predicate[];
  effect?: EffectDescription;
}

// Example: widget synthesis tool
const synthesizeWidgetTool: ToolContract = {
  name: 'synthesize-widget',
  description: 'Generate a new widget FSM and template from a specification',
  input: {
    type: 'object',
    properties: {
      name: { type: 'string' },
      initialStates: { type: 'array', items: { type: 'string' } },
      transitions: { type: 'array' },
      serviceInvocations: { type: 'array' },
    },
  },
  output: {
    type: 'object',
    properties: {
      yaml: { type: 'string' }, // YAML FSM definition
      html: { type: 'string' }, // HTML template
      types: { type: 'string' }, // Generated types
    },
  },
  preconditions: [
    'widget name is unique',
    'all referenced services exist',
  ],
};

```

Agents use tool contracts to:

1. **Discover tools:** Query available tools and their capabilities
2. **Check preconditions:** Verify that tool invocation is valid

3. **Reason about effects:** Understand what state changes a tool will make
4. **Compose tools:** Chain multiple tools to achieve a goal

3. Explicit Planning with Checkpoints

Rather than one-shot prompting, agents propose **structured plans**:

```
interface Plan {
  goal: string; // High-level intent
  steps: PlanStep[];
  checkpoints: Checkpoint[]; // Human review points
}

interface PlanStep {
  tool: string;
  input: Record<string, any>;
  expectedOutput: string;
  rationale: string; // Why this step is necessary
}

interface Checkpoint {
  stepIndex: number;
  humanReview: {
    question: string; // What human should verify
    options: string[]; // Accept / Reject / Modify
  };
}

// Example plan
const plan: Plan = {
  goal: 'Add a login widget to the application',
  steps: [
    {
      tool: 'analyze-service',
      input: { serviceName: 'authenticateUser' },
      expectedOutput: 'Service definition with parameter and return t',
      rationale: 'Understand the authentication service interface',
    },
    {
      tool: 'synthesize-widget',
      input: {
```

```

    name: 'login',
    initialStates: ['idle', 'authenticating', 'success', 'error'],
    serviceInvocations: [
      { state: 'authenticating', service: 'authenticateUser' },
    ],
  },
  expectedOutput: 'YAML FSM and HTML template',
  rationale: 'Generate widget scaffolding based on service contracts',
},
{
  tool: 'validate-widget',
  input: { widgetName: 'login' },
  expectedOutput: 'Validation report (success or errors)',
  rationale: 'Verify widget compiles without errors',
},
],
checkpoints: [
  {
    stepIndex: 1,
    humanReview: {
      question: 'Does the generated widget match the intended behavior?',
      options: ['Accept', 'Reject and explain', 'Modify and retry'],
    },
  },
],
];

```

Execution model:

1. Agent proposes plan
2. Human reviews plan (reads goal, rationale for each step, checkpoint questions)
3. Human accepts or rejects the plan (or requests modifications)
4. If accepted, agent executes steps sequentially
5. At each checkpoint, human can halt execution, inspect results, and decide whether to proceed
6. Results and decisions are logged for future reference

4. Operational Self-Knowledge

The host maintains **operational intelligence**: introspection into its own behavior:

```

interface OperationalIntelligence {
  // Performance metrics
  recentCompilations: CompilationResult[];
  failureHistory: Failure[];
  hotspots: {
    filesThatFailOften: string[];
    statesThatCrashOften: FSMState[];
    servicesThatTimeout: ServiceName[];
  };

  // Development patterns
  frequentErrorPatterns: ErrorPattern[];
  commonRefactorings: Refactoring[];
  testCoverageGaps: string[];

  // Agent decision history
  agentSuggestionsAccepted: number;
  agentSuggestionsRejected: number;
  agentMistakes: string[];
}

```

This intelligence is used to:

1. **Predict issues:** "This change is similar to a previous mistake; be careful"
2. **Improve suggestions:** "Users often reject suggestions of this type; try a different approach"
3. **Identify patterns:** "This error pattern appears in 5 similar scenarios; suggest a systematic fix"

5. Self-Documenting Interactions

All interactions are logged as **structured artifacts**:

```

interface InteractionLog {
  timestamp: number;
  type: 'agent-proposal' | 'human-acceptance' | 'human-rejection' | 'human-error';

  // For proposals
  proposal?: {
    goal: string;
  };
}

```

```

    plan: Plan;
    rationale: string;
};

// For acceptances/rejections
decision?: {
    stepIndex?: number;
    reason: string;
    feedback?: string; // Structured feedback for improvement
};

// For tool invocations
toolCall?: {
    tool: string;
    input: any;
    output: any;
    duration: number;
    success: boolean;
};
}

```

These logs are queryable and can be used to:

1. **Understand collaboration patterns:** What types of suggestions do users accept?
2. **Debug agent errors:** Why did the agent make a wrong suggestion?
3. **Improve agent behavior:** Use logs as training data for fine-tuning

Integration Points with UX3

Session State Querying

Agents access application state through well-defined queries:

```

// Get widget definition
const loginWidget = session.query('widget', { name: 'login' });

// Get service contract
const authService = session.query('service', { name: 'authenticateUser' });

```

```
// Get runtime state
const currentState = session.query('state', { widget: 'login' });
const contextData = session.query('context', { widget: 'login' });

// Get history
const recentEvents = session.query('events', { widget: 'login', last: 1000 });
const recentErrors = session.query('errors', { last: 5 * 60 * 1000 });
```

Tool Integration

Agents invoke development tools through typed contracts:

```
// Synthesize a new widget
const result = await session.invokeTool('synthesize-widget', {
  name: 'profileCard',
  description: 'Display user profile with edit button',
});

// Validate all widgets
const validation = await session.invokeTool('validate-all', {});

// Generate tests for a widget
const tests = await session.invokeTool('generate-tests', {
  widget: 'login',
  coverage: 'paths', // Test all state paths
});
```

Replay and Experimentation

Agents can replay sessions to experiment with hypothetical changes:

```
// Record a session
const session = recordSession(widget);

// Hypothetically modify and re-run
const modifiedWidget = { ...widget, guard: 'ctx.user.role === "admin"' };
const replay = replaySession(modifiedWidget, session);

// Compare outcomes
```

```
console.log(session.finalState); // Original  
console.log(replay.finalState); // With modification
```

Collaborative UX Principles

1. Explicitness Over Convenience

Agents prefer explicit plans over implicit magic. Instead of generating code directly, agents propose structured plans that humans can review and steer.

2. Checkpoints Over Autonomy

Rather than autonomous execution, the system favors explicit checkpoints where humans review and approve changes. This maintains control while enabling agent assistance.

3. Feedback as Data

All human decisions (accept, reject, modify) are logged as structured signals. Over time, this feedback shapes agent behavior.

4. Transparency in Reasoning

Agent reasoning is documented: rationales for each step, precondition checks, and alternative approaches are visible to humans.

Research Questions

1. **Plan synthesis:** How should agents generate plans optimized for human review? (Cf. Yao et al. 2023 on ReAct, Wei et al. 2022 on chain-of-thought)
2. **Feedback learning:** How do we use human acceptances/rejections to improve agent suggestions? (Cf. Ziegler et al. 2019 on RLHF)
3. **Collaborative convergence:** When do human-agent teams outperform humans or agents alone? (Cf. Horvitz 1999 on mixed-initiative systems)
4. **Trust calibration:** How do we maintain appropriate levels of human skepticism? (Cf. Bansal et al. 2019 on explanations and user trust)

Architecture

This chapter describes the architecture of a live collaborative host that combines the dev server, runtime tools, plugins, and sessionful state management.

The host architecture has several layers:

- Runtime state layer: manages widget FSM states, plan state, session state, route state, and user intents in a shared model.
- Agentic orchestration layer: exposes explicit plan creation, execution, stepping, cancellation, and node-update operations so goals can be managed as typed workflow state.
- Knowledge layer: connects indexed content, prompt templates, semantic graph metadata, and project artifacts so agents can make grounded proposals.
- Validation layer: ensures proposed changes pass the same schema, security, and localization checks as authored source.
- Collaboration layer: synchronizes user edits, agent proposals, replay traces, and session events across participants in real time.
- Observation layer: provides invocation history, FSM snapshots, route-aware inspection panels, and event replay so the host can explain itself while running.

The central architectural insight is that an LLM-native host should not hide its cognition. It should expose tools, prompts, resources, plans, and traces as inspectable system elements. That exposed layer is the typed cognitive surface through which humans and agents coordinate with the running system.

Human-AI Workflows

This chapter describes the human-AI workflows that a live host supports.

There are four primary workflow modes:

- Goal capture: users express high-level objectives such as improving a flow or adding a capability. The host converts these into structured goals and context for agents.
- Proposal review: agents generate candidate changes, scaffolds, or plan steps and present them alongside provenance, validation status, and risk indicators.
- Co-execution: users and agents jointly execute plans, update widget states, and track progress through a shared session model.
- Steering: users can interrupt, redirect, or reprioritize a live agent workflow rather than waiting for a full turn to complete.

The chapter also covers the role of self-documentation. Every action, accepted proposal, and plan revision becomes part of the host's evidence graph, enabling later queries such as why a transition was added, what assumptions were used, and which tool outputs shaped the result.

Human steering is decisive. Users can approve, modify, reject, queue, block, or steer suggestions, and the host adapts by refining future proposals while preserving explicit intent history.

Formalism and Goal Semantics

This chapter formalizes how goals, plans, and state are represented in a live host.

Goals are treated as first-class entities with properties such as desired outcome, scope, priority, and success criteria. Plans are hierarchical graphs whose nodes can be human tasks, agent actions, tool calls, or widget changes. State is represented as a combination of current FSM state, plan state, and collaborative session context.

The chapter defines the semantics of operational self-knowledge:

- `Knowledge := {artifact, provenance, timestamp, author, context}`
- `Goal := {id, description, owner, status, successCriteria}`
- `Plan := {nodes, dependencies, state, history}`
- `Session := {mode, route, snapshots, invocations, participants}`

These formalisms support explicit reasoning about intent and allow the host to answer queries such as what goal a proposal serves, whether a plan node reflects validated state, and which prior observations justify the next step.

A lightweight math layer can further express these relations in stable semantic form, which suggests a path toward latex-lite formal appendices and machine-checkable interaction semantics.

Prototype Scenarios

This chapter presents prototype scenarios for a live collaborative host.

Scenario 1: Collaborative feature discovery. A user defines a broad goal, the host proposes a plan with steps, and the session captures both human intent and agent reasoning. The proposal is validated against schema and available tool contracts before being applied.

Scenario 2: LLM-assisted widget scaffolding. The host uses prompt templates, indexed knowledge, and project metadata to draft a new widget. The agent annotates the scaffold with state-machine semantics and a review checklist, while the user approves, edits, or steers the result.

Scenario 3: Self-documenting iteration. As a team accepts plan steps, the host records why each decision was made, the associated goal, and the state transitions involved. This creates a queryable history for later auditing and self-improvement.

Scenario 4: Replay-guided correction. When a workflow fails, the host replays the event trace, surfaces the relevant snapshots and tool outputs, and proposes a corrected next plan step. This makes recovery part of the core collaborative experience rather than a separate debugging mode.

Evaluation and Next Steps

This chapter defines evaluation criteria and next steps for the live host concept.

Evaluation metrics include collaboration throughput, proposal acceptance rate, time to goal completion, replay usefulness, steering effectiveness, and the trustworthiness of agent suggestions. The paper also proposes usability studies focused on mixed human-agent workflows, asking participants to complete goals with and without session memory, replay, and typed plan tools.

Next steps include strengthening multi-participant synchronization, enriching the self-knowledge model, and turning more runtime evidence into portable resources that other tools can consume. Another priority is formalizing the relationship between live host state and compile-time artifacts so that goal progress, plan execution, and validation remain aligned.

The chapter concludes that a successful host should make agentic collaboration feel like a natural extension of the development environment rather than a separate advisory layer.